SR_viz_heterogeneous_homogeneous_sparsity_compare

January 5, 2026

Connected to viz (Python 3.11.14)

Heterogeneous vs Homogeneous Sparsity (Window-Matched) Comparison

This script isolates the effect of *within-corpus heterogeneity* by comparing:

- **Homogeneous boundary average**: mean of two homogeneous sparsities that define the target window.
- **Heterogeneous window**: patterns sampled uniformly inside the same window (mean_sparsity=0.5, sparsity_width sets the range).

We compute **Homogeneous Avg - Heterogeneous** for each (network_size, num_patterns) cell and plot two metrics (rows):

1) **Delta success rate (%)**: difference in the fraction of simulations that recover all patterns before spurious retrieval.
2) **Delta iterations**: difference in the mean iteration index when all patterns are first recovered (only over successful runs; failures are NaN).

Two comparison windows are shown (columns):

- Window [0.3, 0.7]: average of homogeneous sparsity {0.3, 0.7} versus heterogeneous width 0.4 (range [0.3, 0.7]).
- Window [0.1, 0.9]: average of homogeneous sparsity {0.1, 0.9} versus heterogeneous width 0.8 (range [0.1, 0.9]).

Interpretation (Homogeneous - Heterogeneous): - Positive delta success means the homogeneous boundary average performs better. - Negative delta success means heterogeneous mixing performs better. - Positive delta iterations means heterogeneous tends to converge faster (fewer iterations), since higher iteration counts are worse.

This window-matched subtraction controls for overall sparsity range, so any deviation highlights whether *mixing sparsities within a single corpus* helps or hurts recovery relative to homogeneous corpora at the range boundaries.

[ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib as mpl
import seaborn as sns
from pathlib import Path
import sys
```

```
# Add scripts directory to path
sys.path.insert(0, str(Path(__file__).parent.parent))

from utils import load_final_results, DATA_DIR
```

[ ]:
```
# CONFIGURATION
# ============================================================================

# Data sources
HOMOGENEOUS_NAME = "SR_sparsity_sleep_small"
HETEROGENEOUS_NAME = "SR_heterogeneous_sparsity_sleep_small"

# Comparison definitions
COMPARISONS = [
    {
        "label": r"[0.3, 0.7] vs $\Delta s = 0.4$",
        "homo_sparsities": [0.3, 0.7],
        "hetero_width": 0.4,
    },
    {
        "label": r"[0.1, 0.9] vs $\Delta s = 0.8$",
        "homo_sparsities": [0.1, 0.9],
        "hetero_width": 0.8,
    },
]

# Plot settings
SAVE_PLOTS = True
OUTPUT_DIR = Path(__file__).parent.parent / "plots"
DPI = 300
```

[ ]:
```
sns.set_style("darkgrid")
sns.set_context("paper", font_scale=1.5)
plt.rcParams['text.usetex'] = True
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.serif'] = ['Times']
plt.rcParams.update({
    'font.size': 20,
    'axes.labelsize': 20,
    'axes.titlesize': 20,
    'xtick.labelsize': 17,
    'ytick.labelsize': 17,
    'legend.fontsize': 20,
    'figure.titlesize': 20,
    'lines.linewidth': 2.5,
    'axes.linewidth': 1.5,
```

```
    'axes.grid': False,
    'font.weight': 'bold'
})
```

[ ]:
```
def get_spaced_indices(a, n, num_ticks=4):
    """Generate evenly spaced indices for tick marks."""
    return np.linspace(a, n, num_ticks, dtype=int)


def _filter_close(df, column, value, atol=1e-8):
    """Filter rows using an isclose match for float values."""
    return df[np.isclose(df[column], value, atol=atol)]


def _pivot_mean(df, metric, all_num_patterns, all_net_sizes):
    """Pivot mean values onto the common grid."""
    pivot = df.pivot_table(
        values=metric,
        index='num_patterns',
        columns='network_size',
        aggfunc='mean'
    )
    return pivot.reindex(index=all_num_patterns, columns=all_net_sizes)


def compute_difference(df_homo, df_hetero, homo_sparsities, hetero_width,
                       metric, all_num_patterns, all_net_sizes):
    """
    Compute (homogeneous boundary average) - (heterogeneous window) for a␣
 ↪metric.
    """
    homo_pivots = []
    for sparsity in homo_sparsities:
        sub = _filter_close(df_homo, 'sparsity', sparsity)
        homo_pivots.append(_pivot_mean(sub, metric, all_num_patterns,␣
 ↪all_net_sizes))

    with np.errstate(all='ignore'):
        homo_avg_values = np.nanmean(
            np.stack([pivot.values for pivot in homo_pivots]),
            axis=0
        )

    homo_avg = pd.DataFrame(
        homo_avg_values,
        index=all_num_patterns,
        columns=all_net_sizes
```

```
    )

    hetero_sub = _filter_close(df_hetero, 'sparsity_width', hetero_width)
    hetero_pivot = _pivot_mean(hetero_sub, metric, all_num_patterns,␣
 ↪all_net_sizes)

    return homo_avg - hetero_pivot


def _safe_abs_max(values):
    """Return max(abs(values)) or 0 if all NaN."""
    if np.all(np.isnan(values)):
        return 0.0
    return float(np.nanmax(np.abs(values)))
```

Load and Preprocess Data

[ ]:
```
print("=" * 70)
print("LOADING DATA")
print("=" * 70)

homo_path = DATA_DIR / "sleep_results" / HOMOGENEOUS_NAME
hetero_path = DATA_DIR / "sleep_results" / HETEROGENEOUS_NAME

df_homo = load_final_results(homo_path)
df_hetero = load_final_results(hetero_path)

print(f"\nHomogeneous data: {len(df_homo)} simulations from {homo_path}")
print(f"  Sparsity values: {sorted(df_homo['sparsity'].unique())}")

print(f"\nHeterogeneous data: {len(df_hetero)} simulations from {hetero_path}")
print(f"  Sparsity widths: {sorted(df_hetero['sparsity_width'].unique())}")

# For unsuccessful recoveries, set first_iter_all_found to NaN
df_homo = df_homo.copy()
df_hetero = df_hetero.copy()
df_homo.loc[df_homo['all_recovered_before_spurious'] == 0,␣
 ↪'first_iter_all_found'] = np.nan
df_hetero.loc[df_hetero['all_recovered_before_spurious'] == 0,␣
 ↪'first_iter_all_found'] = np.nan

# Use only the common grid between homogeneous and heterogeneous results
all_net_sizes = np.sort(np.intersect1d(
    df_homo['network_size'].unique(),
    df_hetero['network_size'].unique()
))
all_num_patterns = np.sort(np.intersect1d(
```

```
    df_homo['num_patterns'].unique(),
    df_hetero['num_patterns'].unique()
))

print(f"\nCommon grid: {len(all_net_sizes)} network sizes x␣
 ↪{len(all_num_patterns)} pattern counts")
```

```
======================================================================
LOADING DATA
======================================================================

Homogeneous data: 5000 simulations from /mnt/c/Users/pauls/Desktop/Work/PhD/Auto
nomous_Recollection_CHN/simulations/main_write_sleep/data/sleep_results/SR_spars
ity_sleep_small
  Sparsity values: [np.float64(0.1), np.float64(0.3), np.float64(0.5),
np.float64(0.7), np.float64(0.9)]

Heterogeneous data: 5000 simulations from /mnt/c/Users/pauls/Desktop/Work/PhD/Au
tonomous_Recollection_CHN/simulations/main_write_sleep/data/sleep_results/SR_het
erogeneous_sparsity_sleep_small
  Sparsity widths: [np.float64(0.0), np.float64(0.2), np.float64(0.4),
np.float64(0.6), np.float64(0.8)]

Common grid: 20 network sizes x 25 pattern counts
```

Compute Window-Matched Differences

[ ]:
```
diff_matrices = {}
vmax_success = 0.0
vmax_iter = 0.0

for col_idx, comparison in enumerate(COMPARISONS):
    homo_sparsities = comparison["homo_sparsities"]
    hetero_width = comparison["hetero_width"]

    diff_success = compute_difference(
        df_homo, df_hetero, homo_sparsities, hetero_width,
        'all_recovered_before_spurious', all_num_patterns, all_net_sizes
    ) * 100.0

    diff_iter = compute_difference(
        df_homo, df_hetero, homo_sparsities, hetero_width,
        'first_iter_all_found', all_num_patterns, all_net_sizes
    )

    diff_matrices[col_idx] = {'success': diff_success, 'iter': diff_iter}
    vmax_success = max(vmax_success, _safe_abs_max(diff_success.values))
    vmax_iter = max(vmax_iter, _safe_abs_max(diff_iter.values))
```

```
vmax_success = max(vmax_success, 1e-6)
vmax_iter = max(vmax_iter, 1e-6)

print(f"\nColor scale ranges:")
print(f"  Delta success rate: -{vmax_success:.1f}% to +{vmax_success:.1f}%")
print(f"  Delta iterations: -{vmax_iter:.0f} to +{vmax_iter:.0f}")
```

```
Color scale ranges:
  Delta success rate: -100.0% to +100.0%
  Delta iterations: -162 to +162
<ipython-input-4-a6cb9034ab4b>:34: RuntimeWarning: Mean of empty slice
  homo_avg_values = np.nanmean(
<ipython-input-4-a6cb9034ab4b>:34: RuntimeWarning: Mean of empty slice
  homo_avg_values = np.nanmean(
```

Plot Heatmaps (Two Windows, Two Metrics)

[ ]:
```
print("\n" + "=" * 70)
print("CREATING VISUALIZATION")
print("=" * 70)

n_cols = len(COMPARISONS)
n_rows = 2  # Row 0: success rate, Row 1: iteration count

cmap_diff = mpl.cm.get_cmap('RdBu_r').copy()
cmap_diff.set_bad(color="lightgrey")

fig_width = max(9, 4.5 * n_cols)
fig, axes = plt.subplots(n_rows, n_cols, figsize=(fig_width, 9),
                         sharex=True, sharey=True, squeeze=False)

im_success = None
im_iter = None

for col_idx, comparison in enumerate(COMPARISONS):
    diff_success = diff_matrices[col_idx]['success']
    diff_iter = diff_matrices[col_idx]['iter']

    # Row 0: Success rate difference
    ax = axes[0, col_idx]
    masked_success = np.ma.masked_invalid(diff_success.values)
    im_success = ax.imshow(
        masked_success,
        cmap=cmap_diff,
        vmin=-vmax_success, vmax=vmax_success,
```

```
        aspect='auto',
        origin='upper'
    )
    ax.set_title(comparison["label"])
    ax.grid(False)

    # Row 1: Iteration difference
    ax = axes[1, col_idx]
    masked_iter = np.ma.masked_invalid(diff_iter.values)
    im_iter = ax.imshow(
        masked_iter,
        cmap=cmap_diff,
        vmin=-vmax_iter, vmax=vmax_iter,
        aspect='auto',
        origin='upper'
    )
    ax.grid(False)

# Set ticks
x_tick_indices = get_spaced_indices(0, len(all_net_sizes) - 1, 4)
y_tick_indices = get_spaced_indices(0, len(all_num_patterns) - 1, 7)

for row in axes:
    for ax in row:
        ax.tick_params(axis='both', which='both', bottom=True, left=True,
                       top=False, right=False)
        ax.set_xticks(x_tick_indices)
        ax.set_xticklabels(all_net_sizes[x_tick_indices])
        ax.set_yticks(y_tick_indices)
        ax.set_yticklabels(all_num_patterns[y_tick_indices])

# Add colorbars (manual placement to avoid overlap)
cbar1_ax = fig.add_axes([0.90, 0.56, 0.02, 0.30])
cbar1 = fig.colorbar(im_success, cax=cbar1_ax)
cbar1.set_label(r'$\Delta$ Success rate (\%)')

cbar2_ax = fig.add_axes([0.90, 0.14, 0.02, 0.30])
cbar2 = fig.colorbar(im_iter, cax=cbar2_ax)
cbar2.set_label(r'$\Delta$ Iterations')

# Axis labels
fig.text(0.5, 0.06, 'Network size', ha='center', va='center')
fig.text(0.06, 0.5, 'Nb stored patterns', ha='left', va='center', rotation=90)

# Leave room for labels and colorbars
plt.tight_layout(rect=[0.08, 0.08, 0.88, 0.98])
```

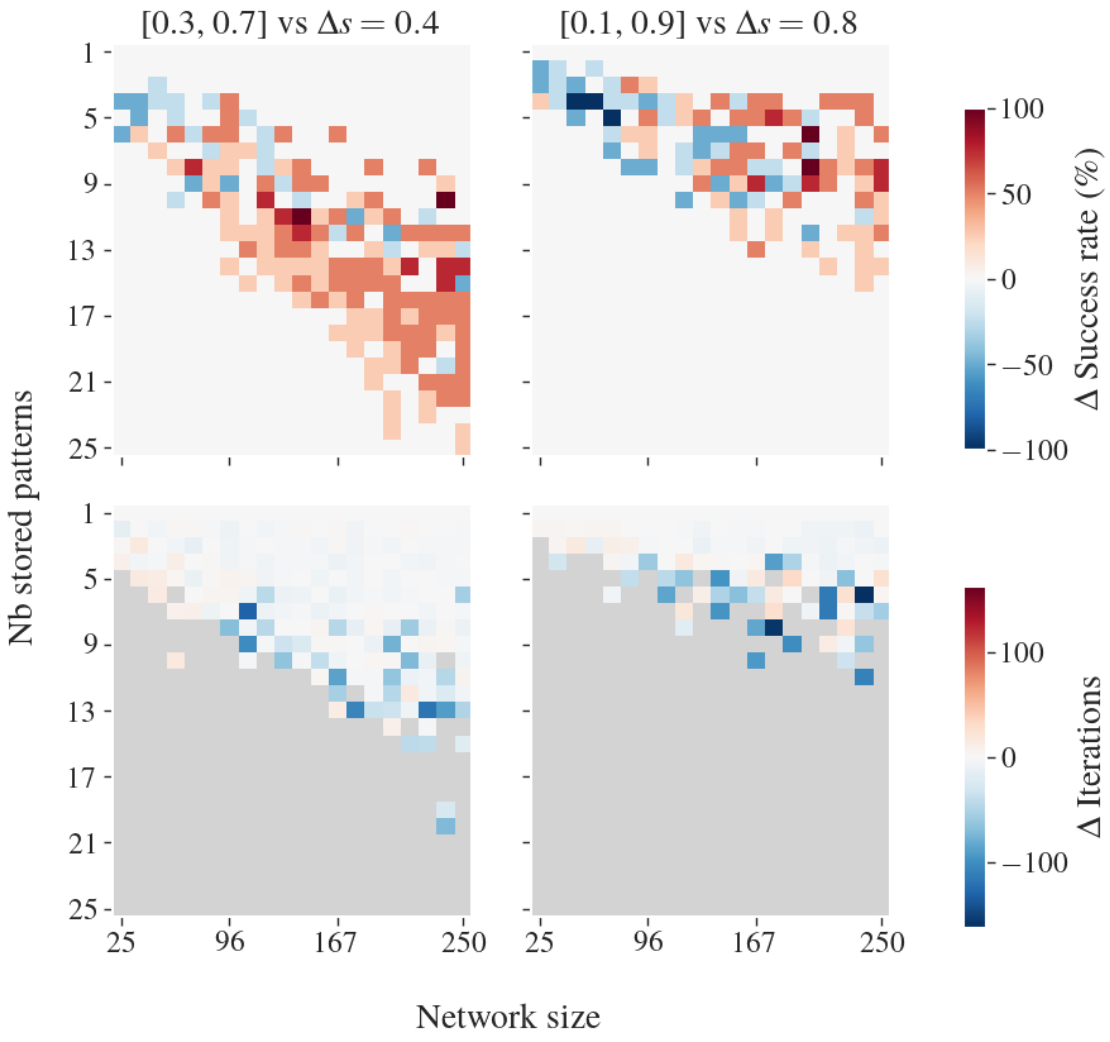
```
if SAVE_PLOTS:
    OUTPUT_DIR.mkdir(exist_ok=True, parents=True)
    output_path = OUTPUT_DIR / "SR_homogeneous_vs_heterogeneous_window_compare.
 ↪png"
    fig.savefig(output_path, dpi=DPI, bbox_inches='tight')
    print(f"\nSaved figure to: {output_path}")

plt.show()
```

```
<ipython-input-7-9081699ddb8a>:9: MatplotlibDeprecationWarning: The get_cmap
function was deprecated in Matplotlib 3.7 and will be removed in 3.11. Use
``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap()`` or
``pyplot.get_cmap()`` instead.
  cmap_diff = mpl.cm.get_cmap('RdBu_r').copy()
<ipython-input-7-9081699ddb8a>:75: UserWarning: This figure includes Axes that
are not compatible with tight_layout, so results might be incorrect.
  plt.tight_layout(rect=[0.08, 0.08, 0.88, 0.98])


======================================================================
CREATING VISUALIZATION
======================================================================

Saved figure to: /mnt/c/Users/pauls/Desktop/Work/PhD/Autonomous_Recollection_CHN
/simulations/main_write_sleep/scripts/plots/SR_homogeneous_vs_heterogeneous_wind
ow_compare.png
```



[ ]:
```
print("\n" + "=" * 70)
print("SUMMARY STATISTICS")
print("=" * 70)

for col_idx, comparison in enumerate(COMPARISONS):
    label = comparison["label"]
    diff_success = diff_matrices[col_idx]['success']
    diff_iter = diff_matrices[col_idx]['iter']

    mean_diff_success = np.nanmean(diff_success.values)
    mean_diff_iter = np.nanmean(diff_iter.values)

    print(f"\n{label}:")
    print(f"  Mean delta success rate: {mean_diff_success:+.2f}%")
    print(f"  Mean delta iterations: {mean_diff_iter:+.1f}")
```

```python
print("\n" + "=" * 70)
print("VISUALIZATION COMPLETE!")
print("=" * 70)
```

```
======================================================================
SUMMARY STATISTICS
======================================================================

[0.3, 0.7] vs $\Delta s = 0.4$:
  Mean delta success rate: +8.45%
  Mean delta iterations: -10.1

[0.1, 0.9] vs $\Delta s = 0.8$:
  Mean delta success rate: +1.90%
  Mean delta iterations: -16.1

======================================================================
VISUALIZATION COMPLETE!
======================================================================
```

No kernel connected

Connected to viz (Python 3.11.14)